

Programy użytkowe

2025

Na wykładzie (i ćwiczeniach) będziemy zapoznawać się z narzędziami/programami przydatnymi na studiach i pracy fizyka.

Co fizyk robi?

- Wykonuje pomiary $\Rightarrow$ analizuje je $\Rightarrow$ buduje teorie (analitycznie i numerycznie) $\Rightarrow$ weryfikuje je.

Co fizyk robi?

- Wykonuje pomiary $\Rightarrow$ analizuje je $\Rightarrow$ buduje teorie (analitycznie i numerycznie) $\Rightarrow$ weryfikuje je.
- Czynności:
 - Przetwarzanie danych
 - Obliczenia analityczne
 - Symulacje
 - Wykonanie wykresów
 - Prezentacja wyników – publikacje

Program wykładu

- Przetwarzanie danych – Octave
- Obliczenia analityczne – Maxima
- Symulacje – Bash
- Wykresy – Gnuplot
- Publikacje – Latex, (Word)

- Przetwarzanie danych – Octave
 - Obliczenia analityczne – Maxima
 - Symulacje – Bash
 - Wykresy – Gnuplot
 - Publikacje – Latex, (Word)
1. Bash
 2. Octave
 3. Gnuplot
 4. Latex
 5. Maxima

Wprowadzenie do systemu Linux

-  Sobell M. G. 2013 Linux. Programowanie w powłoce. Praktyczny przewodnik. Wydanie III (Helion)
-  Sosna Ł. 2021 Linux. Komendy i polecenia. Wydanie V (Helion)
-  Barrett D. J. 2012 Linux. Leksykon Kieszonkowy. Wydanie II (Helion)
-  Schroder C. 2009 Sieci Linux: receptury (Helion)

Bash

Bash – Co to jest?

Z Wikipedii:

Jest domyślną powłoką w większości dystrybucji systemu GNU/Linux oraz w systemie Mac OS X od wersji 10.3, istnieją także wersje dla większości systemów uniksowych.

Bash jest domyślną powłoką w środowisku Cygwin dla systemów Win32.

Bash umożliwia pracę interaktywną i wsadową.

Język Basha umożliwia definiowanie aliasów, funkcji, zawiera konstrukcje sterujące przepływem (if, while, for, ...).

Powłoka bash zachowuje historię wykonywanych poleceń.

Obecnie dostępny jest Bash 5.1.8

Skąd i kto?

Nazwa jest akronimem od Bourne-Again Shell (fonetycznie brzmi tak samo, jak born again shell).

Wywodzi się od powłoki Bourne'a sh, która była jedną z pierwszych i najważniejszych powłok systemu UNIX.

Bash był pisany głównie przez Briana Foksa i Cheta Rameya w 1987.

Pod systemem Windows dostępny jest [CygWin](#)

W wersji elektronicznej:

<http://www.emarcel.com/emarcel/linux-articles/45-kurs-bash.html>

http://www.linux.org.pl/PLUG/susebook/tekst_suzie_3c.pdf

i inne.

Tradycyjnej:



Elen Siever, “Linux Podręcznik Użytkownika”, Wydawnictwo RM, Warszawa 1999



C. Albing, J. Vossen “Bash. Receptury”, Helion 2021



S. Lakshman “Skrypty powłoki systemu Linux. Receptury”, Helion 2012



J. Drózd “Skrypty w Shellu. Programowanie w powłoce Bash”
Mikom 2005

Dlaczego?

Prawdy oczywiste.

- Im większy komputer tym większe prawdopodobieństwo pracy “na konsoli”.
- Im dalej jest komputer tym szybsza jest praca przez konsolę.
- Do pracy nie jest potrzebny graficzny interfejs użytkownika!
- Łatwość automatyzacji pracy! – przetwarzania dużej ilości plików, bądź “kaskadowego” uruchamiania programów.

Minimalny skrypt

```
#!/bin/bash  
echo "Witamy Panstwa"
```

Dodatkowo należy nadać skryptowi atrybut wykonywalności.
Bardzo dobrym zwyczajem jest komentowanie zawartości skryptu
(programu) służy do tego znak #, czyli:

Minimalny skrypt cd.

```
#!/bin/bash  
echo "Witamy Panstwa" # wyświetlenie tekstu na ekranie
```

Zmienne

Deklaracja

Jeżeli zmienna jest liczbą lub ciągiem liter bez **spacji**:

ZMIENNA=1234

ZMIENNA=tekst

gdy występują spacje to stosujemy cudzysłów:

ZMIENNA="tekst przykładowy"

Zmienne można zadeklarować w dowolnym dla programisty momencie.

Nie wymagają podania typu.

Po obu stronach znaku "=" nie mogą znajdować się spacje!

Dobry zwyczaj: ZMIENNE PISZEMY DUŻYMI LITERAMI.

Odwołanie

\$ZMIENNA

Do oznaczenia zmiennych stosuje się znak cudzysłowia i apostrofu.
Wykonanie skryptu:

```
#!/bin/bash  
ZMIENNA="Przykładowy tekst"  
echo "$ZMIENNA"  
echo '$ZMIENNA'
```

da wynik:

```
Przykładowy tekst  
$ZMIENNA
```

czyli zmienna wewnątrz apostrofów została potraktowana jako tekst.

Problem: jak do zmiennej dopisać nazwę bez spacji? Np:

ZMIENNA=90

chcemy wyświetlić: 90stopni

Stosujemy wtedy nawiasy "wąsate" {}

```
#!/bin/bash
```

```
ZMIENNA=90
```

```
echo "${ZMIENNA}stopni"
```

Do pobrania zmiennych służy instrukcja `read`.

```
#!/bin/bash
echo "Podaj swoje imie:"
read IMIE
echo "Na imie masz: $IMIE"
```

Parametry

Parametry są bardzo wygodnym sposobem przekazania zmiennych do skryptu, jednakże nie mogą być już później zmienione. Odwołanie się do parametrów odbywa się przez ich numer porządkowy.

- `$*` – wszystkie parametry począwszy od parametru numer 1
- `$0` – nazwa programu
- `$1` – parametr nr 1
- ...
- `$9` – parametr nr 9
- `${nn}` parametr nr nn (tylko w stanie wyższej konieczności)
- `$#` – liczba parametrów
- `$$` – numer procesu

skrypt1:

```
#!/bin/bash
echo "nazwa skryptu: $0"
echo "liczba parametrow: $#"
```

echo "wszystkie parametry wywołania: \$*"
echo "parametr pierwszy: \$1"
echo "parametr drugi: \$2"
echo "parametr trzeci: \$3"
echo "numer procesu: \$\$"

Wywołanie tego skryptu w postaci:

```
./skrypt1 123 3 456 678
```

```
skrypt1
```

```
4
```

```
123 3 456 678
```

```
123
```

```
3
```

```
456
```

```
678
```

Operacje arytmetyczne

Na zmiennych można też wykonywać proste operacje matematyczne:

```
#!/bin/bash
echo "Podaj liczbę:"
read L1
echo "Podaj jeszcze jedna liczbę:"
read L2
SUMA=$(( $L1 + $L2 ))
echo "suma tych liczb wynosi: $SUMA"
```

Do wykonywania operacji arytmetycznych używa się operatora **expr**.

```
WYNIK_DZIALANIA='expr 13 - 7'
```

Dostępne operatory:

dodawanie(+), odejmowanie (-), mnożenie (*), dzielenie (/) i reszta z dzielenia (%).

W Bashu dostępne są dwie instrukcje warunkowe: **case**, **if**.

Instrukcja kończy się zapisaniem słowa kluczowego od końca!

if → *fi*

case → *esac*

Instrukcje warunkowe: if

Tak jak w większości języków programowania instrukcja if przewiduje trzy scenariusze:

- Sprawdzamy warunek $\Rightarrow$ w przypadku powodzenia wykonujemy pewien ciąg instrukcji.
- Sprawdzamy warunek $\Rightarrow$ w przypadku powodzenia wykonujemy pewien ciąg instrukcji a niepowodzenia inny.
- W przypadku niepowodzenia konieczne jest sprawdzenia dodatkowego warunku i w zależności od jego spełnienia wykonanie pewnych czynności. (W zasadzie jest to “case” z dwoma przypadkami.)

if I przypadek

```
if warunek; then  
    wyrażenie1;  
    wyrażenie2;  
fi
```

if II przypadek

```
if warunek; then  
    wyrażenie1;  
else  
    wyrażenie2;  
fi
```

```
if warunek; then  
    wyrażenie1;  
elif warunek2; then  
    wyrażenie2;  
fi
```

Ogólna struktura:

[wyrażenie operand wyrażenie]

Bardzo ważne!

Pomiędzy nawiasami, wyrażeniami i operandem muszą być spacje!

Uwaga:

Czasami do określenia warunku wystarczy podanie wartości wyrażenia – czyli jednego argumentu.

Operatory instrukcji if (wybrane)

Jednoargumentowe:

- -n – *wyrażenie* ma niezerową długość
- -z – *wyrażenie* ma zerową długość
- -d – istnieje katalog o nazwie *wyrażenie*
- -f – sprawdza czy *wyrażenie* jest plikiem
- -e – sprawdza czy zbiór (plik, katalog) o nazwie *wyrażenie* istnieje
- -L – sprawdzenie czy plik o nazwie *wyrażenie* jest dowiązaniem symbolicznym

Operatory instrukcji if (wybrane)

Dwuargumentowe:

- `-eq` – sprawdza czy *wyrażenia* są równymi liczbami
- `-neq` – sprawdza czy *wyrażenia* są różnymi liczbami
- `=` – sprawdza czy *wyrażenia* są jednakowymi ciągami znaków
- `!=` – sprawdza czy *wyrażenia* są różnymi ciągami znaków
- `-lt` – sprawdza czy *wyrażenie 1* jest mniejsze od *wyrażenie 2* (**dla liczb całkowitych**)
- `-le` – sprawdza czy *wyrażenie 1* jest równe lub mniejsze od *wyrażenie 2* (**dla liczb całkowitych**)
- `-gt` – sprawdza czy *wyrażenie 1* jest większe od *wyrażenie 2* (**dla liczb całkowitych**)
- `-ge` – sprawdza czy *wyrażenie 1* jest równe lub większe od *wyrażenie 2* (**dla liczb całkowitych**)

Jak to działa?

Jeżeli zmienna zawiera spację, lub może być pusta, to przy jej wywołaniu należy zastosować cudzysłów.

```
#!/bin/bash
ZMIENNA=""
if [ -n $ZMIENNA ]; then
# -n sprawdza czy argument nie jest pusty
    echo " Zmienna o nazwie ZMIENNA nie jest pusta";
else
    echo " Zmienna o nazwie ZMIENNA jest pusta";
fi
```

da jako wynik:

Zmienna o nazwie ZMIENNA nie jest pusta

Jak to działa?

oczekiwany wynik otrzymamy w przypadku skryptu:

```
#!/bin/bash
ZMIENNA=""
if [ -n "$ZMIENNA" ]; then
    echo " Zmienna o nazwie ZMIENNA nie jest pusta"
else
    echo " Zmienna o nazwie ZMIENNA jest pusta"
fi
```

da oczekiwany wynik:

Zmienna o nazwie ZMIENNA jest pusta.

Przykład 11

```
#!/bin/bash
if [ $1 -eq $2 ]; then
    echo "liczby $1 i $2 sa rowne";
else
    echo "liczby $1 i $2 sa rozne";
fi
if [ -e $3 ]; then
    echo "plik o nazwie $3 istnieje";
    if [ -L $3 ]; then
        echo "i jest dowiązaniem symbolicznym";
    elif [ -f $3 ]; then
        echo "i jest zwykłym plikiem";
    fi
else
    echo "plik o nazwie $3 nie istnieje"
fi
```

Instrukcja case

W Bashu dostępne są dwie instrukcje warunkowe: **case**, **if**.
Składnia tej instrukcji wygląda następująco:

```
case $ZMIENNA in
    1) zadanie numer 1;;
    2) zadanie numer 2;;
    *) zadanie w każdym innym przypadku;;
esac
```

Instrukcja kończy się zapisaniem słowa kluczowego od końca!

Przykład (10):

```
#!/bin/bash
case $1 in
piatek) echo "weekend jest juz tuz tuz";;
sobota) echo "juz weekend";;
niedziela) echo "weekend się juz powoli konczy";;
poniedziałek) echo "już poniedziałek trzeba isc do pracy";;
*) echo "srodek tygodnia trzeba ciezko pracowac";;
esac
```

Pętla for

Składnia:

```
for ZMIENNA in ...; do
    instrukcje;
done
```

Przykład (12):

```
#!/bin/bash
for ZMIENNA in $*; do
    echo "$ZMIENNA";
done
```

lub (13)

```
#!/bin/bash
for ZMIENNA in `ls`; do
    echo "$ZMIENNA";
done
```

Pętla for

Ważne: Pętla for przebiega listę parametrów!
Parametry te można podać "ręcznie". (14)

```
#!/bin/bash
DZIEN1=poniedziałek
DZIEN2=wtorek
DZIEN3=środa
for ZMIENNA in "$DZIEN1" "$DZIEN2" "$DZIEN3"; do
    echo "Dzisiaj jest: $ZMIENNA";
done
```

Pętla **for** wykonuje instrukcje zawarte wewnątrz pętli na parametrach, które są oddzielone spacjami dlatego też zaleca się umieszczanie tych parametrów cudzysłowach.

Pętla for

W przypadku gdy pętlę **for** chcemy użyć do *klasycznej* iteracji to możemy posłużyć np:

z wykorzystanie systemowego polecenia `for i in 'seq 1 10'`

Uwaga: stosujemy *odwrotne apostrofy*.

składni C `for ((i=1, i<11, i++))`

Uwaga: instrukcja ta nie zawsze działa – zależy od implementacji Basha.

Oczywiście nie są to jedyne rozwiązania problemu ...

Pętla while

Składnia:

```
while warunek; do  
    instrukcje;  
done
```

W pętli *while* warunek ma taką samą składnię jak w przypadku instrukcji warunkowej *if*.

Pętla *while* działa tak długo dopóki warunek jest prawdziwy w przeciwnym razie pętla kończy swoje działanie.

Do sterowania pętlą *while* istnieją dwie instrukcje:

- **break** – przerwanie wykonywania pętli
- **continue** – wymuszenie kolejnej iteracji pętli

Pętla while

Przykład (15):

```
#!/bin/bash
X=$1
while [ $X -gt $2 ]; do
    ROZNICA=$(( $X - $2 ));
    echo "liczba $X jest wieksza od liczby $2 o $ROZNICA"
    X=$(( $X - 1 ));
done
```

Pętla until

Pętla *until* jest w zasadzie “negatywem” pętli *while*.
Jest ona wykonywana tak długo aż warunek petli stanie się prawdziwy. Składnia:

```
until warunek; do
    instrukcje;
done
```

Przykład (16):

```
#!/bin/bash
X=$1
until [ $X -gt $2 ]; do
    echo "liczba $X jest mniejsza od liczby $2";
    X=$((X + 1));
done
```

Znaki globalne

Wszystkie parametry zawierające znak “*” zastępowane są listą plików, które pasują do zadanego wzorca.

Sam znak “*” jest zastępowany listą wszystkich plików znajdujących się w bieżącym katalogu za wyjątkiem plików, których nazwa zaczyna się od kropki.

Przykład (17):

```
#!/bin/bash
echo "Oto zawartość katalogu ${PWD}:"
echo ${PWD}/*
```

Gwiazdka wykorzystana do utworzenia maski (18):

```
#!/bin/bash
echo "Pliki tekstowe w katalogu ${PWD}:"
echo ${PWD}/*.txt
```

- `> plik` Przekierowuje standardowe wyjście do pliku
- `< plik` Pobiera dane wejściowe z pliku
- `>> plik` Kieruje standardowe wyjście do *plik*; jeżeli plik istnieje to dane są do niego dopisywane.
- `<> plik` Używa pliku zarówno do standardowego wejścia jak i wyjścia.

Mechanizm podstawiania poleceń pozwala na wykonanie polecenia linuksowego i traktowaniu jego wyniku tak jakby to były dane wprowadzone “z klawiatury”.

Podstawianie poleceń można wprowadzić na dwa sposoby:

- rozwijanie zawartości nawiasów,
- ujmowanie polecenia w tzw. wsteczne apostrofy.

Rozwijanie zawartości nawiasów: \$(polecenie_do_wykonania) – zapis taki zostaje zastąpiony przez wynik wykonania polecenia znajdującego się w nawiasach. Nawiasy mogą być zagnieżdżone, czyli polecenia również mogą zawierać nawiasy.

Ujmowanie polecenia w tzw. wsteczne apostrofy:

'polecenie_do_wykonania' – zapis taki zostaje zastąpiony przez wynik wykonania polecenia znajdującego się w apostrofach.

Przykład (19):

```
#!/bin/bash
DANE="$(ls -l)"
echo "Wynik metody rozwijania zawartosci nawiasow:"
echo "$DANE"
DANE=`cat test.txt`
echo "Wynik metody ujmowania polecenia w tzw. wsteczne a"
echo "$DANE"
```

Funkcje

Składnia:

```
nazwa_funkcji()
```

```
{
```

```
instrukcje do wykonania
```

```
}
```

Przykład:

```
dodawanie()
```

```
{
```

```
    WYNIK=$(( $1 + $2 ))
```

```
    echo "$WYNIK"
```

```
}
```

W przetwarzaniu dużej ilości plików bardzo pomocne są metaznaki:

- * – zastępuje każdy ciąg znaków
- ? – zastępuje pojedynczy znak
- [abc...] zastępuje znak jednym z podanych w nawiasie, też z podanego zakresu np. a-z, A-Z, 0-9
- [!abc...] zastępuje znak jednym ze znaków nie podanym w nawiasie

polecenie & – wykonuje polecenie w tle

polecenie1 ; *polecenie2* – wykonuje ciąg poleceń w jednej linii

polecenie1 | *polecenie2* – używa danych wyjściowych z *polecenie1* jako dane wejściowe *polecenie2*

polecenie1 ' *polecenie2* ' – uzupełnianie się poleceń, używa danych wyjściowych z *polecenie1* jako argumentów *polecenie2*

polecenie1 && *polecenie2* – wykonanie *polecenie2* jeżeli *polecenie1* zakończyło się sukcesem

polecenie1 || *polecenie2* – wykonanie *polecenie2* jeżeli *polecenie1* zakończyło się porażką

Operatory zastępujące

- `${zmienna:-słowo}` Jeżeli zmienna istnieje i nie jest pusta to zwraca jej wartość w przeciwnym wypadku zwraca słowo.
Zastosowanie: chcemy sprawdzić, czy dana zmienna istnieje.
- `${zmienna:=słowo}` Jeżeli zmienna istnieje i nie jest pusta to zwraca jej wartość w przeciwnym wypadku ustawia jej wartość na słowo i wtedy zwraca jej wartość. Zastosowanie: ustalamy wartość domyślną.
- `${zmienna:?wiadomosc}` Jeżeli zmienna istnieje i nie jest pusta to zwraca jej wartość w przeciwnym wypadku wyświetla zmienną i jej wiadomość i przerywa wykonanie danego polecenia lub skryptu.
- `${zmienna:+słowo}` Jeżeli zmienna istnieje i nie jest pusta to zwraca jej słowo w przeciwnym wypadku zwraca zero.
Zastosowanie: testowanie czy zmienna istnieje.

Operatory wzorców dopasowania

- `${zmienna#worzec}` Jeśli wzorzec pasuje do początku wartości zmiennej, usuwa najkrótszą pasującą część i zwraca resztę.
- `${zmienna##worzec}` Jeśli wzorzec pasuje do początku wartości zmiennej, usuwa najdłuższą pasującą część i zwraca resztę.
- `${zmienna%worzec}` Jeśli wzorzec pasuje do końca wartości zmiennej, usuwa najkrótszą pasującą część i zwraca resztę.
- `${zmienna%%worzec}` Jeśli wzorzec pasuje do końca wartości zmiennej, usuwa najdłuższą pasującą część i zwraca resztę.

Użyteczne polecenia

- **basename** – usuwa ścieżkę dostępu i przyrostek z podanej nazwy.
- **cat** – odczytuje jeden lub więcej plików i drukuje je na standardowym wyjściu. Wykorzystywany do łączenia plików.
- **column** – formatuje dane wejściowe w kolumny wypełniając najpierw rzędy; -x – lub kolumny.
- **cut** – wycina zaznaczone kolumny lub pola z jednego lub większej liczby plików.
- **nohup** – kontynuuje wykonywanie polecenia nawet po wylogowaniu się użytkownika.
- **paste** – łączy odpowiadające sobie linie jednego lub więcej plików w pionowe kolumny oddzielone znakiem tabulacji

Dodatkowe informacje

Więcej o dopasowaniach

`${ZMIENNA%.txt}`

`moj_plik.txt` → `moj_plik`

?

`proces_123.txt` → `123`

`TMP=${ZMIENNA%.txt}`

`${TMP#proces_}`

`proces_123.txt` → `proces_123`

`proces_123` → `123`

Rozszerzone dopasowania

Standardowe dopasowania

- `*` 0 lub więcej znaków (dowolnych)
- `?` 1 znak (dowolny)
- `[...]` znak ze zbioru

Możemy używać `(wzorzec1|wzorzec2|...)`

Wymagana komenda: `shopt -s extglob`

Rozszerzone dopasowania

- `?(wzorzec)` 0 lub 1 wystąpienie wzorca
- `*(wzorzec)` 0 lub więcej wystąpień wzorca
- `+(wzorzec)` 1 lub więcej wystąpień wzorca
- `@(wzorzec)` 1 wystąpienie wzorca

Co, jeśli nie znamy przedrostka i zakończenia?

nieznany_przedrostek_123456332_nieznane_zakończenie
→ 123456332

```
ZMIENNA=przedrostek_12232.rozszerzenie  
TMP=${ZMIENNA2#+([ ^0-9])}  
echo ${TMP%+([ ^0-9])}
```

```
${ZMIENNA/WZORZEC/ZAMIANA/}  
${ZMIENNA//WZORZEC/ZAMIANA/}  
${ZMIENNA/#WZORZEC/ZAMIANA/}  
${ZMIENNA/%WZORZEC/ZAMIANA/}
```

Poprzedni przykład w jednej linie

```
echo "${ZMIENNA//+([!0-9])}"
```

nieznany_przedrostek_123456332_nieznane_zakończenie

→ 123456332

```
ls -lh | while read line
do
    echo $line
done
```

```
while read line
do
    echo $line !
done < <(ls -lh)
```